Lab 3 Report – Vitis HLS Directive Analysis

EE19B108
Vikas Raj

Objective

The objective of this experiment was to analyze how Vitis HLS directives influence scheduling, hardware resource usage, and performance when synthesizing a differential equation solver. Different directive configurations were applied to observe how pipelining and resource allocation constraints affect the generated hardware architecture.

All designs were synthesized targeting the Zynq xc7z020-clg400-2 FPGA.

Experiment Setup

Four synthesis solutions were created with different directive configurations.

Solution 1 – Default Implementation

No directives were applied. The HLS tool used its default scheduling and optimization strategy.

Solution 2 – Disable Pipelining and Limit Multipliers

#pragma HLS PIPELINE off

#pragma HLS ALLOCATION operation instances=mul limit=1

This configuration disables loop pipelining and restricts the number of multipliers to one instance.

Solution 3 – Pipeline with Multiplier Limit

#pragma HLS PIPELINE II=1

#pragma HLS ALLOCATION operation instances=mul limit=1

The design attempts to achieve **Initiation Interval (II) = 1** while still limiting the multiplier resource.

Results and Observations

Default Design

In the default configuration, Vitis HLS automatically allocates multiple arithmetic units and schedules operations to maximize performance. The tool creates parallel multipliers to execute operations simultaneously.

Observation

- Higher DSP usage due to parallel multipliers
- Improved performance and throughput
- Hardware architecture optimized automatically by the tool

Resource-Limited Design (Pipeline Off)

When pipelining is disabled and multipliers are limited to one, the loop executes sequentially. All multiplication operations must share the same hardware multiplier.

Observation

- DSP usage significantly reduced
- Operations scheduled sequentially
- Increased execution latency compared to the default design

This configuration demonstrates the trade-off between resource usage and performance.

Pipeline with Multiplier Limit

With `PIPELINE II=1`, the design attempts to start a new iteration every clock cycle. However, since only one multiplier is available, multiple multiplication operations must share the same hardware resource.

Observation

- Resource sharing occurs
- The requested II = 1 cannot always be achieved due to multiplier limitation
- Performance improves compared to the non-pipelined design but remains constrained by hardware resources

---

Key Findings

1. Default HLS synthesis prioritizes performance by generating parallel arithmetic units.
2. Resource allocation directives reduce hardware usage by forcing operations to share functional units.
3. Disabling pipelining increases latency because loop iterations execute sequentially.
4. Strict resource limits prevent achieving very low initiation intervals.
5. Relaxing the clock constraint allows the scheduler to handle resource sharing and complete synthesis successfully.